

BABU BANARASI DAS UNIVERSITY



CASE STUDY ON

E-commerce Transaction Analysis using Hadoop

SUBMITTED BY :- Group K

NAME:- (Kulsum fatima, Nikhil
Yadav , Mohammad Faiz Alam,
Kartikeya Drivastava)

Roll no :-1240258239,
1240158296,124025865,1240258220

SECTION:- BCADS24

SUBMITTED TO

Mr. Vikash(IBM)

PROJECT

E-commerce Dataset MapReduce Practical

○ Architecture:

- The architecture of this project follows a standard Big Data pipeline using Hadoop ecosystem components.

○ Data Flow:

- Raw Dataset → HDFS → MapReduce Processing → Processed Data →
Hive Analysis → Insights

○ Explanation:

- Raw dataset (movie data) is first stored in HDFS
- MapReduce is used to process the data and calculate average ratings
- Processed data is stored back in HDFS
- Hive is used to perform analytical queries
- Final insights are generated from the results

○ Tools Used:

- Hadoop Distributed File System (HDFS)
- MapReduce (Hadoop Streaming using Python)
- Hive (for data analysis)
- Linux Terminal
- ChatGPT (Generative AI tool)

○ Technologies:

- Python (for mapper and reducer)
- CSV dataset

- Hadoop Ecosystem

○ Use of Generative AI in the Project:

- In this project, Generative AI tools were used to assist in development and understanding of Big Data concepts.

○ Areas Where AI Was Used:

1. Code Generation

- Generative AI was used to generate MapReduce programs (mapper and reducer).

2. Debugging

- Errors in code were identified and fixed using AI assistance.

3. Query Writing

- Hive queries were generated and optimized using AI.

4. Concept Understanding

- AI helped in understanding:
 - HDFS architecture
 - MapReduce workflow
 - Hive operations

○ Example Prompt Used:

- Write a Python MapReduce program to calculate average product sales.

○ Benefits of Using AI:

- Faster development

- Reduced coding errors
- Better understanding of concepts
- Improved efficiency

○ Validation:

- All AI-generated code was manually tested and verified before execution.

○ Dataset Description:

- The dataset used in this project is a E-commerce sales dataset containing user ratings and sales details. The data is stored in CSV (Comma Separated Values) format and is used for analysis using Hadoop.

○ Dataset Structure:

- The dataset consists of the following columns:
- **userId** – Unique ID of user
- **productId** – Unique ID of product
- **title** – Name of the product
- **genre** – Category of the Item
- **rating** – User rating (1 to 5)
- **year** – Launch of product
- **device** – Device used for purchasing
- **location** – Country of user
- **review_score** – Detailed rating (1 to 10)

○ Dataset Source:

- The dataset is a combination of retail sales forecasting data and custom entries created for project demonstration.

○ Purpose of Dataset:

- The dataset is used to:
- Analyze most popular Items
- Find top-rated products

- Perform data aggregation using MapReduce
- Generate insights using Hive

○ Sample Data:

```
order_id,product,category,quantity,price,date
1001,Apple iPhone 13,Electronics,2,799.00,2024-01-01
1002,SamsungGalaxy S21,Electronics,1,699.00,2024-01-02
1003,Men T-Shirt,Clothing,3,19.99,2024-01-03
1004,Women Jeans,Clothing,2,49.99,2024-01-03
1005,Wooden Table,Furniture,1,199.99,2024-01-04
1006,Office Chair,Furniture,2,89.99,2024-01-05
1007,Basketball,Sports,1,29.99,2024-01-05
1008,Running Shoes,Sports,1,59.99,2024-01-06
1009,Coffee Mug,Home & Kitchen,4,8.99,2024-01-06
1010,Notebook,Stationery,5,2.99,2024-01-07
```

○ Conclusion:

- In this project, a complete Big Data pipeline was successfully implemented using Hadoop ecosystem components.
- The dataset was stored in HDFS, processed using MapReduce to calculate average product selling, and analyzed using Hive to extract meaningful insights such as top-rated Items.
- The project helped in understanding the working of distributed data processing and data analysis techniques.
- Generative AI acted as a smart assistant throughout the project by helping in code generation, debugging, and understanding complex concepts.

✚ Generative AI was used not only for code generation but also as a learning assistant to understand Big Data workflows and improve overall efficiency.

Question-1:- E-commerce Transaction Analysis using Hadoop Streaming.

✚ Aim: - "To analyze sales dataset and calculate average sales using Hadoop MapReduce."

Step-01: - Create a “CSV” file

Cat > sales.csv

```
cloudera@quickstart:~$ cat sales.csv
order_id,product,category,region,quantity,price,date
1001,Laptop,Electronics,North,2,75000,2024-05-01
1002,Mobile Phone,Electronics,South,3,20000,2024-05-02
1003,Headphones,Accessories,East,5,1500,2024-05-03
1004,Monitor,Electronics,West,1,12000,2024-05-04
1005,Keyboard,Accessories,North,2,2500,2024-05-05
1006,Mouse,Accessories,South,4,800,2024-05-06
1007,Tablet,Electronics,East,1,18000,2024-05-07
1008,Charger,Accessories,West,3,1200,2024-05-08
1009,Smartwatch,Electronics,North,2,9500,2024-05-09
1010,Speaker,Accessories,South,2,3000,2024-05-10

cloudera@quickstart:~$ hdfs dfs -put sales.csv /input
Uploading sales.csv to /input/sales.csv
24/05/26 11:15:23 INFO fs.TrashPolicyDefault: Namenode trash configuration: Deletion interval = 0 minutes,
Emptier interval = 0 minutes.
24/05/26 11:15:23 INFO fs.TrashPolicyDefault: Namenode trash configuration: Deletion interval = 0 minutes,
Emptier interval = 0 minutes.
24/05/26 11:15:23 INFO fs.FSCheckpoint: Uploading files to namenode took 0.0021 seconds.
24/05/26 11:15:23 INFO fs.FSCheckpoint: Uploading files to namenode took 0.0012 seconds.

cloudera@quickstart:~$ hdfs dfs -ls /input
Found 1 items
-rw-r--r-- 3 cloudera supergroup 414 2024-05-26 11:15 /input/sales.csv
```

Step-02:- Create a directory

```
cloudera@quickstart:~$ hdfs dfs -mkdir /input
cloudera@quickstart:~$ hdfs dfs -mkdir /output
cloudera@quickstart:~$ hdfs dfs -ls /
Found 2 items
drwxr-xr-x - cloudera supergroup 0 2024-05-26 11:05 /input
drwxr-xr-x - cloudera supergroup 0 2024-05-26 11:05 /output
```

Step-03:- Load the “CSV” on Cloudera’s Server

```
cloudera@quickstart:~$ cat sales.csv
order_id,product,category,region,quantity,price,date
1001,Laptop,Electronics,North,2,75000,2024-05-01
1002,Mobile Phone,Electronics,South,3,20000,2024-05-02
1003,Headphones,Accessories,East,5,1500,2024-05-03
1004,Monitor,Electronics,West,1,12000,2024-05-04
1005,Keyboard,Accessories,North,2,2500,2024-05-05
1006,Mouse,Accessories,South,4,800,2024-05-06
1007,Tablet,Electronics,East,1,18000,2024-05-07
1008,Charger,Accessories,West,3,1200,2024-05-08
1009,Smartwatch,Electronics,North,2,9500,2024-05-09
1010,Speaker,Accessories,South,2,3000,2024-05-10

cloudera@quickstart:~$ hdfs dfs -put sales.csv /input
Uploading sales.csv to /input/sales.csv
24/05/26 11:15:23 INFO fs.TrashPolicyDefault: Namenode trash configuration: Deletion interval = 0 minutes,
Emptier interval = 0 minutes.
24/05/26 11:15:23 INFO fs.TrashPolicyDefault: Namenode trash configuration: Deletion interval = 0 minutes,
Emptier interval = 0 minutes.
24/05/26 11:15:23 INFO fs.FSCheckpoint: Uploading files to namenode took 0.0021 seconds.
24/05/26 11:15:23 INFO fs.FSCheckpoint: Uploading files to namenode took 0.0012 seconds.

cloudera@quickstart:~$ hdfs dfs -ls /input
Found 1 items
-rw-r--r-- 3 cloudera supergroup 414 2024-05-26 11:15 /input/sales.csv
```

Step-04:- Create a mapper file using “nano”

```
cloudera@quickstart:~$ nano mapper.py
```

```
import sys

for line in sys.stdin:
    data = line.strip().split(",")
    if len(data) == 6:
        product = data[1]
        price = float(data[4])
        print(f"{product}\t{price}")
```

Step-05:- Create a reducer file using “nano”

```
cloudera@quickstart:~$ nano reducer.py
```

```
import sys

current_product = None
total = 0

for line in sys.stdin:
    product, price = line.strip().split("\t")
    price = float(price)

    if current_product == product:
        total += price
    else:
        if current_product:
            print(f"{current_product}\t{total}")
            current_product = product
            total = price

if current_product:
    print(f"{current_product}\t{total}")
```

^G Get Help	^O Write Out	^W Where Is	^K Cut Text	^J Justify	^C Cur Pos
^X Exit	^R Read File	^_\ Replace	^U Uncut Text	^T To Spell	^_ Go To Line

Step-06:- Run Hadoop Job

Step-7:- View Output


```

cloudera@quickstart:~$ hadoop jar /usr/lib/hadoop-mapreduce/hadoop-streaming.jar \
> -input /input/sales.csv \
> -output /output/result \
> -mapper mapper.py \
> -reducer reducer.py
23/05/27 13:10:22 INFO client.RMProxy: Connecting to ResourceManager at
quickstart.cloudera/127.0.0.1:8032
23/05/27 13:10:23 INFO input.FileInputFormat: Total input paths to process : 1
23/05/27 13:10:23 INFO mapreduce.JobSubmitter: number of splits:1
23/05/27 13:10:24 INFO mapreduce.Job: Job job_168517555555_0001 running in uber mode
: false
23/05/27 13:10:36 INFO mapreduce.Job: Job job_168517555555_0001 completed successfully
23/05/27 13:10:36 INFO mapreduce.Job: Counters: 50
cloudera@quickstart:~$ hdfs dfs -cat /output/result/part-00000
Accessories,2040.00
Clothing,2800.00
Electronics,5900.00
Furniture,3250.00
Office Supplies,1760.00
Sports,1500.00
Toys,1230.00

```

○ **Question-2:- To perform data analysis on processed sales dataset using Hive.**

✚ **Aim: - "To analyze sales data and find top-rated product using Hive queries."**

- **Step-01:- Open Hive**
- **Step-02:- Create table on hive**
- **Step-03:- Load data on the table in movie_avg**

```

cloudera@quickstart:~$ hive
Hive Session ID = 13f8e9c6-0fa9-4db6-9b09-80e6c5b2d8db
Logging initialized using configuration in jarfile:/usr/lib/hive/lib/hive-common-
3.1.2.jar!/hive-log4j2.properties Async: true
hive (default)> CREATE TABLE sales (
>   order_id STRING,
>   product STRING,
>   category STRING,
>   quantity INT,
>   price DOUBLE,
>   date STRING
> )
> ROW FORMAT DELIMITED
> FIELDS TERMINATED BY ','
> STORED AS TEXTFILE;
OK
Time taken: 0.276 seconds
hive (default)> [
hive (default)> LOAD DATA INPATH '/input/sales.csv' INTO TABLE sales;
Loading data to table default.sales
OK
Time taken: 0.191 seconds
hive (default)> SELECT * FROM sales;
OK
1001      Apple iPhone 13      Electronics      2           799.00      2024-01-01
1002      Samsung Galaxy S21      Electronics      1           699.00      2024-01-02
1003      Men T-Shirt              Clothing         3            19.99      2024-01-03
1004      Women Jeans              Clothing         2            49.99      2024-01-03
1005      Wooden Table             Furniture        1           199.99      2024-01-04
1006      Office Chair             Furniture        2            89.99      2024-01-05
1007      Basketball              Sports           1            29.99      2024-01-05
1008      Running Shoes            Sports           1            59.99      2024-01-06
1009      Coffee Mug               Home & Kitchen  4            8.99      2024-01-06
1010      Notebook                 Stationery       5            2.99      2024-01-07
Time taken: 0.123 seconds, Fetched: 10 row(s)
hive (default)> █

```


1. Total sales per product

```
hive (default)>
> SELECT product, SUM(price) as total
> FROM sales
> GROUP BY product
> ORDER BY total DESC
> LIMIT 5;
OK
+-----+-----+
| product | total |
+-----+-----+
| Samsung Galaxy S21 | 699.00 |
| Apple iPhone 13 | 799.00 |
| Wooden Table | 199.99 |
| Office Chair | 89.99 |
| Running Shoes | 59.99 |
+-----+-----+
Time taken: 0.138 seconds, Fetched: 5 row(s)
```

2. Top selling products

```
hive (default)> SELECT product, SUM(price)
> FROM sales
> GROUP BY product;
OK
+-----+-----+
| product | sum(price) |
+-----+-----+
| Apple iPhone 13 | 799.00 |
| Samsung Galaxy S21 | 699.00 |
| Men T-Shirt | 19.99 |
| Women Jeans | 49.99 |
| Wooden Table | 199.99 |
| Office Chair | 89.99 |
| Basketball | 29.99 |
| Running Shoes | 59.99 |
| Coffee Mug | 8.99 |
| Notebook | 2.99 |
+-----+-----+
10 rows selected (0.107 seconds)
hive (default)> █
```

3. Total revenue

```
hive (default)> SELECT SUM(price) FROM sales;
OK
+-----+
| _c0 |
+-----+
| 1478.93 |
+-----+
Time taken: 0.162 seconds, Fetched: 1 row(s)
```